

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Applied Sciences · Department of Computer Science

Solution Book 6

SCP 1101, August 2004 Supplementary

SCS 1101 Introduction to Computer Science and Programming

Past-paper solutions with marks allocated, algorithms, textbook flowcharts, exam-ready code and a civil & water engineering lens

Supplementary paper · 5 in total, at least 2 per section · page 1 (Section A) supplied

Book 6 of 8 · Prepared October 2026

Index — tap a tab or a line to jump

Every chapter starts with the coloured tab bar below; tap any tab to jump straight to that question. The page footer has a link back to this index. In Word, the Navigation Pane shows the same chapters.

Guide **How to use this book**

Glance **At a glance**

Q1 **Question 1 [20]**

- ▸ Five printing devices: operation, impact or non-impact, quality and speed [4 each]

Q2 **Question 2 [20]**

- ▸ 2(i) Explain these terms [2 each = 10]
- ▸ 2(ii) Operating system: definition and five functions [10]

Q3 **Question 3 [20]**

- ▸ 3(i) Advantages of high-level languages over low-level languages [5]
- ▸ 3(ii) Compare first- and fourth-generation computers [15]

Refs **Sources for this book**

Links **Source register**

How to use this book

This book is one of eight stand-alone solution books for NUST SCS 1101 past papers. It solves **SCP 1101, August 2004 Supplementary** question by question, with the marks the examiner allocates shown in square brackets, e.g. [2].

The six-block answer template

Every answer in this book (and in all eight books of the set) is built from the same six blocks, in the same order. Theory answers use blocks 1, 2, 5 and 6; programming answers use all six.

1. **Like you're 3** — one or two sentences with a toy, a bath or a water tap.
2. **Model answer** — the exam wording, with the mark for each point in brackets, e.g. [2]. Write it like this and you collect every mark.
3. **Algorithm + code** — numbered pseudocode first, then C code that follows the **H-D-I-P-O-R** recipe.
4. **Flowchart or structure chart** — textbook (ANSI) symbols. Programs built from functions also get a structure (hierarchy) chart.
5. **Engineering lens** — the same idea in a civil and water engineering setting, using the profession's vocabulary.
6. **Examiner's tip** — where students usually lose marks.

The H-D-I-P-O-R code recipe — "**H**appy **D**ucks **I**n **P**onds **O**nly **R**elax". Every exam program is these six steps, in this order:

```c
/* H - Header: what the program does, and the libraries */
#include <stdio.h>

int main(void)
{
    /* D - Declare: every variable, with a sensible type and a starting value */
    /* I - Input:   printf a prompt, then scanf with & */
    /* P - Process: the if-ladder, loop or formula */
    /* O - Output:  printf the result with units and decimals */
    /* R - Return:  */ return 0;
}
```

The recipe is worth marks by itself: examiners give separate marks for headers, declarations, input, processing and output. A half-finished program laid out this way still scores; a clever one-liner without structure often does not.

Algorithm style used in every book:

```
Step 1: Start
Step 2: Read <inputs>
Step 3: <process, one action per step>
Step 4: Display <outputs>
Step 5: Stop
```

Flowchart symbols (ANSI):

Symbol	Shape	Use	Colour in this book
Terminator	Rounded box (stadium)	Start / Stop	Green

Symbol	Shape	Use	Colour in this book
Input/Output	Parallelogram	Read or display	Blue
Process	Rectangle	Calculation or assignment	Amber
Decision	Diamond	Yes/no test, two exits	Red
Predefined process	Rectangle with double sides	Call to a function	Purple
Flow line	Arrow	Order of steps	Dark grey arrow

The smallest possible program, drawn with these symbols:

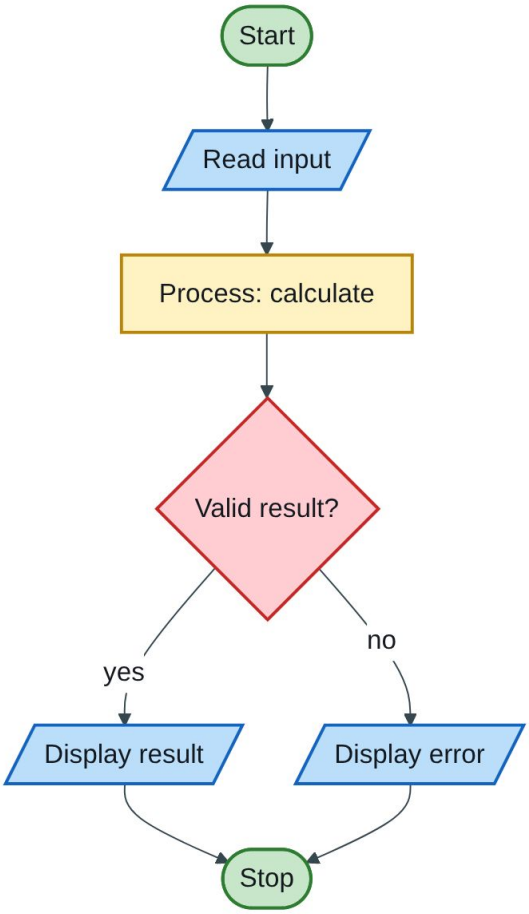


Figure 6.1 — Flowchart

The engineering lens. Programming questions get a civil and water engineering version where it fits, for example:

- tariff ladders become **increasing block tariffs** for water;
- array statistics become **rainfall** or **concrete cube strength** analysis;
- geometry programs become **cross-section, wetted perimeter and hydraulic radius** calculations.

Write the exam version in the exam; the engineering version is for understanding and for your later modules.

How the solutions were researched

Every fact a solution relies on is traced to a source in one of four tiers; each source has a short key (e.g. [MIT-2]) so answers can cite it without long links.

1. **Tier 1 — the course itself.** The SCS 1101 outline and the past papers decide *what* to answer and how marks are split.

2. **Tier 2 — university teaching material.** MIT OpenCourseWare and UCLA course pages for programming and algorithms; university histories for milestones.
3. **Tier 3 — standards and research.** NIST for cloud definitions; peer-reviewed and arXiv papers, including Wits research on cloud adoption in South Africa.
4. **Tier 4 — standard textbook knowledge.** Definitions every textbook agrees on (OS functions, topologies, printers) are written from the textbooks your outline lists, without a web citation.

A claim is only cited when a source was actually opened; anything from tier 4 stays general and uncited.

At a glance

SCP 1101 Introduction to Computer Science, August 2004 supplementary. 3 hours, 20 marks per question; answer **5 questions in total, at least 2 from each section**. Only **page 1 (Section A, Q1–Q3)** was provided, so this book solves those three; Section B (programming) is missing.

All three questions are theory, so marks come from complete, structured comparisons: use tables.

Question	What it tests	Marks
Q1	Five printers: operation, impact or not, quality, speed	20
Q2	System software, EPROM, primary memory, register, virtual memory; OS functions	10 + 10
Q3	Advantages of high-level languages; 1st vs 4th generation	5 + 15

Question 1 [20]

Five printing devices: operation, impact or non-impact, quality and speed [4 each]

Like you're 3: impact printers **hit** the paper through an inky ribbon, like a stamp; non-impact printers **spray, melt or draw** without hitting.

For each printer: how it works [2], impact or non-impact [1], quality and speed [1].

Printer	How it works	Impact?	Quality	Speed (approximate)
(a) Daisy wheel	A plastic or metal wheel has a fully formed character on each "petal"; the wheel spins to the right letter and a hammer strikes it against the ribbon and paper.	Impact	High ("letter quality") text, but no graphics	Slow: tens of characters per second
(b) Dot matrix	A print head with 9 or 24 pins moves across the page; selected pins strike the ribbon, forming characters from dots.	Impact	Low to medium (draft or near-letter quality); can do carbon copies	Medium: hundreds of characters per second
(c) Inkjet	Tiny nozzles spray droplets of ink onto the paper, using heat (bubble jet) or piezo-electric crystals.	Non-impact	Good, including colour photos	Moderate: a few to about 20 pages per minute
(d) Laser	A laser beam draws the page as an electrostatic image on a drum; toner powder sticks to it, transfers to the paper and is fused by heat.	Non-impact	Very high, sharp text	Fast: a whole page at a time, tens of pages per minute
(e) Plotter	Pens (or inkjet heads) are moved over large paper under computer control to draw continuous lines.	Non-impact (pen or inkjet)	Very accurate line drawings on A1/A0 sheets	Slow per sheet, but precise

Engineering lens: civil engineers still print **A1 and A0 construction drawings** on large-format plotters (now mostly inkjet), while site offices keep a laser printer for reports and certificates.

Question 2 [20]

2(i) Explain these terms [2 each = 10]

Like you're 3: system software is the school's caretaker; EPROM is a notebook you can wipe only with a special sun lamp; primary memory is your desk; a register is the scrap of paper in your hand; virtual memory is borrowing floor space when the desk is full.

Term	Explanation [1 + 1]
System software	Programs that run and manage the computer itself and support application software. Examples: operating systems, device drivers, utilities, compilers.
EPROM	Erasable Programmable Read-Only Memory: non-volatile memory that can be erased by **ultraviolet light** through a window on the chip and then reprogrammed. Used for firmware during development.
Primary memory	The main memory the CPU accesses directly (RAM, ROM and cache), holding the programs and data in use; fast but limited in size.
Register	A very small, very fast storage location **inside the CPU** that holds the current instruction, data or address, e.g. the program counter and accumulator.
Virtual memory	A technique where the OS uses part of the **hard disk as an extension of RAM**, swapping pages in and out, so programs larger than physical RAM can run (more slowly).

2(ii) Operating system: definition and five functions [10]

Definition [2]: the operating system is the system software that manages the computer's hardware and software resources, provides services to programs, and acts as the interface between the user and the hardware. Examples: Windows, Linux.

Five functions [8]: process management (scheduling programs on the CPU); **memory management** (allocating RAM, virtual memory); **file management** (organising and protecting files); **device management** (drivers, spooling to printers); **security** (log-ins, permissions); **user interface** (GUI or command line). Name and explain any five.

Question 3 [20]

3(i) Advantages of high-level languages over low-level languages [5]

Like you're 3: a high-level language lets you talk to the computer in near-English ("add the tax to the price"), instead of in its own baby-talk of 0s and 1s.

Any five, 1 each:

1. **Easier to learn, read and write** — English-like words and maths symbols (`if`, `while`, `total = a + b`).
2. **Faster to develop** — one statement replaces many machine instructions.
3. **Portable** — the same source code can be compiled for different CPUs; machine code is tied to one CPU family.
4. **Easier to debug and maintain** — meaningful names, comments and structure; compilers report errors with line numbers.
5. **Problem-oriented** — the programmer thinks about the engineering problem, not registers and memory addresses.
6. **Rich libraries** — ready-made functions for maths, files and graphics.

3(ii) Compare first- and fourth-generation computers [15]

Like you're 3: a first-generation computer was a room-sized giant made of hot glass tubes; a fourth-generation computer is a laptop or phone built around one tiny chip.

3 marks per heading: one fact for each generation plus the comparison.

Heading	1st generation (about 1940–1956)	4th generation (1971 onwards)
(a) Storage devices	Magnetic drums, mercury delay lines, early magnetic-core memory; punched cards and paper tape for bulk data	Semiconductor RAM in gigabytes; hard disks, SSDs, flash drives, optical discs, cloud storage
(b) Hardware components	Thousands of **vacuum tubes**; filled whole rooms; huge power use and heat	**Microprocessor**: VLSI chips with millions to billions of transistors; desktop, laptop and handheld size
(c) I/O devices	Punched-card readers, paper tape, switches, teleprinter printouts	Keyboard, mouse, touchscreen, scanner, high-resolution monitor, laser and inkjet printers
(d) Programming languages	Machine language (binary) and early assembly	High-level languages (C, C++, Java, Python), 4GLs such as SQL, and GUIs
(e) Efficiency	Slow (thousands of operations per second), unreliable (tubes burned out), very expensive	Billions of operations per second, very reliable, cheap and energy-efficient

A good first-generation example is MIT's **Whirlwind**, a vacuum-tube machine that introduced reliable random-access magnetic-core memory (MIT-3); a typical fourth-generation example is any PC built around an Intel or AMD microprocessor.

Examiner's tip: keep each comparison on the **same line** for both generations (a table does this); examiners can then tick the comparison, not just the facts.

Sources for this book

- [MIT-3] Whirlwind Computer, 1944–59 (IEEE Milestone) — first-generation example (Q3).
- [NUST-1] SCS 1101 course outline.
- Textbook knowledge (tier 4): printer technology, memory types, virtual memory, OS functions, language levels and computer generations. Printer speeds are approximate ranges.

Source register

The full register for the solution-book set. Sources marked ✓ are cited in this book; the rest (MIT, UCLA and Wits material used across the set) are further reading on the same topics.

Key	Source	Institution	Used for	In this book
NUST-1	SCS 1101 course outline (uploaded)	NUST	Syllabus, assessment, C as the course language	✓
MIT-1	6.087 Practical Programming in C	MIT	C syntax, pointers, structs, files, coding style	Further reading
MIT-2	6.006 Lecture 19: Dynamic Programming I (Fibonacci)	MIT	Why naive recursive Fibonacci is slow; memoization	Further reading
MIT-3	Whirlwind Computer, 1944–59 (IEEE Milestone)	MIT	Magnetic-core memory, real-time computing	✓
MIT-4	Wang et al., *Undefined Behavior: What Happened to My Code?*	MIT CSAIL	Run-time and undefined-behaviour errors	Further reading
UCLA-1	First internet message, 29 Oct 1969	UCLA Samueli	History of networks and the internet	Further reading
UCLA-2	Birthplace of the Internet (IEEE Milestone)	UCLA	ARPANET, packet switching, first node	Further reading
UCLA-3	CS 31 Introduction to Computer Science I, syllabus	UCLA	Intro-programming scope; portability across compilers	Further reading
WITS-1	Sithole (Wits) & Ruhode, *Cloud Computing Adoption: Opportunities and Challenges for SMMEs in South Africa*, arXiv:2108.10079	Wits	Cloud benefits and barriers in Southern Africa	Further reading

Key	Source	Institution	Used for	In this book
WITS-2	*Leadership and the adoption of cloud computing for enterprises in Johannesburg* (2022)	Wits	Drivers and barriers of cloud adoption	Further reading
STD-1	NIST SP 800-145, *The NIST Definition of Cloud Computing*	NIST	SaaS, PaaS, IaaS; 5 characteristics; 4 deployment models	Further reading
RES-1	Bakasa & Pekane, *Decision Criteria for Cloud Adoption by Large Enterprises in South Africa*, arXiv:2108.09757	UCT	Security and data sovereignty as selection criteria	Further reading
RES-2	Fu et al., *Error Log Analysis in C Programming Language Courses* (ICCE 2015)	Kyushu University	The errors novice C students actually make	Further reading